

a) Lectura de robots.txt: ¿puede un agente acceder al sitio?

Este ejercicio usa `urllib.robotparser.RobotFileParser`, de la librería estándar de Python, para interpretar el archivo `robots.txt` de la tienda virtual publicada en `http://localhost:3000` y determinar si un *user-agent* determinado tiene permiso para acceder a una URL concreta.

Flujo de trabajo:

- `set_url(url)` : define la ubicación del `robots.txt`.
- `read()` : descarga y procesa el archivo, cargando las reglas en memoria.
- `can_fetch(user_agent, url)` : devuelve `True / False` según las directivas `Allow / Disallow`.

Documentación oficial: <https://docs.python.org/3/library/urllib.robotparser.html>

```
from urllib.robotparser import RobotFileParser

BASE_URL = "http://localhost:3000"
USER_AGENT = "MineriaWeb-2026-2/1.0"

parser = RobotFileParser()
parser.set_url(f"{BASE_URL}/robots.txt")
parser.read()

print(f"Sitemaps declarados: {parser.site_maps()}")
```

Sitemaps declarados: ['http://localhost:3000/sitemap.xml']

Verificando el acceso a distintas rutas

El `robots.txt` de la tienda virtual (`app/robots.txt`) permite todo el sitio (`Allow: /`) salvo dos rutas de edición:

```
User-Agent: *
Allow: /
Disallow: /productos/*/editar
Disallow: /clientes/*/editar
```

Probemos `can_fetch` contra rutas que deberían estar permitidas y contra las dos que deberían estar bloqueadas.

```
rutas_a_verificar = [
    "/",
    "/productos",
    "/productos/1",
    "/productos/1/editar",
```

```

"/clientes",
"/clientes/1/editar",
"/testimonios",
"/ordenes",
]

for ruta in rutas_a_verificar:
    permitido = parser.can_fetch(USER_AGENT, f"{BASE_URL}{ruta}")
    print(f"{ruta:25s} -> {'PERMITIDO' if permitido else 'BLOQUEADO'}")

/ -> PERMITIDO
/productos -> PERMITIDO
/productos/1 -> PERMITIDO
/productos/1/editar -> BLOQUEADO
/clientes -> PERMITIDO
/clientes/1/editar -> BLOQUEADO
/testimonios -> PERMITIDO
/ordenes -> PERMITIDO

```

Una limitación importante de `urllib.robotparser`

Las rutas `/productos/1/editar` y `/clientes/1/editar` deberían aparecer como **BLOQUEADO**, pero `RobotFileParser` las marca como **PERMITIDO**.

La razón es que `urllib.robotparser` (a diferencia de librerías como `protego` o de los rastreadores de Google/Bing) **no interpreta el comodín `*` dentro de una ruta**: compara la URL contra el patrón `/productos/*/editar` como si el asterisco fuera un carácter literal, y como ninguna URL real contiene un `*`, la regla nunca coincide.

Esto es una lección práctica valiosa: la librería estándar es útil para reglas simples, pero para *robots.txt* con comodines conviene interpretar las reglas manualmente o usar una librería especializada. A continuación implementamos una verificación manual que sí soporta comodines, traduciendo cada patrón a una expresión regular.

```

import re

DISALLOW = ["/productos/*/editar", "/clientes/*/editar"]

def coincide_con_patron(patron: str, path: str) -> bool:
    """Traduce un patron de robots.txt (con * y $) a una regex y verifica
    coincidencia."""
    regex = re.escape(patron).replace(r"\*", ".*")
    if regex.endswith(r"\$"):
        regex = regex[:-2] + "$"
    return re.match(regex, path) is not None

def accesible_para_agente(path: str) -> bool:
    return not any(coincide_con_patron(patron, path) for patron in DISALLOW)

for ruta in rutas_a_verificar:
    permitido = accesible_para_agente(ruta)
    print(f"{ruta:25s} -> {'PERMITIDO' if permitido else 'BLOQUEADO'}")

```

/	-> PERMITIDO
/productos	-> PERMITIDO
/productos/1	-> PERMITIDO
/productos/1/editar	-> BLOQUEADO
/clientes	-> PERMITIDO
/clientes/1/editar	-> BLOQUEADO
/testimonios	-> PERMITIDO
/ordenes	-> PERMITIDO

Con la verificación manual, `/productos/1/editar` y `/clientes/1/editar` sí aparecen como **BLOQUEADO**, coincidiendo con la intención real del archivo `robots.txt`. El resto de ejercicios de esta sesión (b–e) sólo visitan rutas de lectura (`/productos/...`, `/clientes`, `/testimonios`, `/ordenes/...`), por lo que están permitidas para nuestro agente.